Diagnose Before Solving: AI Simulation

Scope Document (v1.2)

Project owner: David Bacon
Document status: Revised for publication
Date: August 31, 2026 (v1.0); revised November 11, 2026 (v1.1) and November 12, 2026 (v1.2)
Supersedes: devlin.ai-hosted version of *Diagnose Before Solving*

> **Status of this document.** From this point forward this scope document is the working source of truth for the project. The original planning document (*Diagnose Before Solving Planning Doc.pdf*) was a private starting point, not a portfolio artifact; it is retained as a reference to consult when a novel question arises that this document does not answer.

> **Revision note (v1.1, November 11, 2026).** The pre-simulation training is no longer a Storyline module. It is a separate static site on GitHub Pages that links to the simulation (D-173, D-177). The simulation's embed capability was kept as a verified feature rather than removed (D-188). This revision changes the wording of sections 4.1, 4.2, 5, 6.3, 7, 8 (criterion 6), 9, 10.1 and 11 so that Storyline is one possible host among others, and so that embedding is verified with a plain test page (the embed harness, Test Plan section 9.1) instead of a published Storyline module. The simulation itself, sections 1 to 3, and the rubric are unchanged. Part 1 of the decision log is frozen and still describes the Storyline plan as it was decided at the time.

> **Revision note (v1.2, November 12, 2026).** Corrects a factual error in section 6.3: an unset `frame-ancestors` directive does not block framing, it permits it (D-200, D-203). Records what is now settled: the training's link carries `?embed=1` (D-190), the host is Vercel and the models are Sonnet 5 and Opus 5 (Technical Design 4 and 5), and the embed harness is built (D-191). Section 6.2, the latency row of the risk table, section 10 and section 11 change accordingly. Nothing in the simulation's content or rubric changes. A later pass on the same day corrected two statements that said demo mode covers any API failure (section 4.3 and the model-deprecation risk row): it covers budget and storage failures at session start, and a mid-call model error returns a retryable message (D-144).

---

1. Purpose

1.1 Why this project exists

Diagnose Before Solving is an AI-powered conversation simulation in which a learner plays an internal tech support specialist taking an angry call from a regional sales lead. The learner practices diagnosing the real problem before proposing a fix.

The simulation already exists as a portfolio artifact hosted on devlin.ai. That hosting has failed in the field: a prospective employer attempted to open the simulation and could not, because the devlin.ai account credits had been exhausted. A portfolio artifact that intermittently does not load is worse than no artifact at all, because the failure happens in front of exactly the audience it was built for.

This project rebuilds the simulation as a self-hosted application under the project owner's direct control.

1.2 Goals

1. **Reliability.** The simulation loads and runs for any visitor, at any time, with no dependency on a third-party platform's credit balance or continued existence.

2. **Fidelity.** The learning design carries over intact from the original: same character, same scenario, same behavior triggers, same scoring rubric, same feedback model.
3. **Demonstrated capability.** The rebuild itself becomes portfolio evidence. A hiring manager should see not only a working simulation but the design reasoning behind it: scope, decisions, prompt architecture, evaluation rubric, and testing.
4. **Cost predictability.** Running the simulation must never produce a surprise bill, and hitting a spending ceiling must never produce a broken page.

1.3 Non-goals

This is not a commercial product, not a platform, and not a general-purpose simulation authoring tool. It is one well-built scenario. Design decisions that trade generality for a better single experience are correct decisions here.

---

2. Audience

2.1 Primary audience: hiring managers and recruiters

The people this artifact is actually built for are L&D hiring managers, instructional design leads, and technical recruiters reviewing the portfolio at dbacon6093.github.io/LDPortfolio.

Design implications:

- They will spend two to five minutes, not twenty. The experience must be legible immediately.
- They may not complete the simulation. Value must be visible before the first message is sent.
- They are evaluating the *designer*, not the *learner experience* in isolation. The supporting documents matter as much as the simulation.
- They will arrive from a link, possibly on mobile, possibly with no context.

2.2 Secondary audience: the simulated learner

Within the fiction of the simulation, the learner is a junior-to-mid internal tech support specialist assigned to support a sales organization. Target competencies: emotional acknowledgment before troubleshooting, open-ended diagnostic questioning, distinguishing a workflow problem from a technical defect, and appropriate escalation.

2.3 Tertiary audience: the project owner

Future maintenance is done by one person with strong instructional design skill, passing familiarity with front-end web work, and limited backend development experience. Architecture choices must favor readability and low maintenance over sophistication.

---

3. The simulation content

This section is the authoritative scope inventory for what the simulation contains. It is not the full design specification; the design and prompt architecture document expands it into the actual instructions given to the model.

Title: Diagnose Before Solving
Language: English
Modality in scope: Text chat

3.1 AI character

Marcus (he/him), regional sales lead, six years managing a team of eight reps. Sharp, direct, takes team performance personally. Convinced a recent CRM update wrecked his team's momentum. Opens the call at a boiling point.

Marcus's emotional arc is the spine of the simulation: he starts hot, softens as the learner demonstrates understanding, hardens if the learner dismisses or deflects, and ends grateful if genuinely heard.

Opening line (Marcus speaks first): "Finally. Look, this update to the CRM is tanking my team's numbers. Can we please get this fixed?"

3.2 Scenario

A CRM update added a new required input field early in the sales workflow. Reps frequently do not have the information the field asks for at that stage. Consequences: less experienced reps stall, and other reps enter fabricated data to move leads forward, corrupting the CRM record. Three of Marcus's eight reps missed stage-progression targets this month.

The field belongs much later in the workflow. The root cause is workflow design, not a software defect and not a training gap. This distinction is the entire learning point.

What Marcus understands of this when the call begins. He has fragments and a suspicion, not a diagnosis. He knows the drop-off started right after the update, he has looked in the CRM himself and seen incomplete and inaccurate data in the new required field, and he suspects that field is the cause. He does not know whether it is a defect or a deliberate design. He has heard secondhand that the workflow changed, and that another support specialist told one of his reps the software was working as intended.

He called before he understood his own problem, which is the same error the learner is being trained out of. The full knowledge model, and what he will and will not say, is specified in the design and prompt architecture document, section 3.

3.3 Learner role

Internal tech support specialist assigned to the sales team, taking Marcus's inbound call.

3.4 Behavior triggers (8 in scope)

The system must detect and respond to these learner moves:

#	Learner move	Marcus's response	Direction
1	Jumps to troubleshooting or asks for a ticket number before acknowledging frustration	Cuts the learner off; demands to know why IT rolled out the update without consulting sales	Escalating
2	Acknowledges the drop in numbers as a real business problem (commissions, team impact), not just a tech inconvenience	Relief; "I'm glad you get it. So how are you going to fix the problem?"	De-escalating
3	Asks a specific, open-ended question about what reps experience at the new field	Shifts into detail mode; describes reps guessing at values or entering fake data	Neutral, opens up
4	Suggests this is a training problem or that reps should read the release notes	Sharp and frustrated; "This isn't a training problem, it is a software issue YOU created."	Escalating
5	Raises escalation to the CRM product owner or CRM lead	Softens noticeably and engages more warmly. When the learner also gives a clear next step and timeline, Marcus settles fully, agrees to the plan, asks to be kept in the loop, and wants resolution by end of week	De-escalating
6	Implies the issue is out of IT's hands and cannot be fixed	Escalates; threatens to go to the COO or VP of Sales	Escalating
7	Suggests the sales team should change their workflow to fit the new software	More frustrated and hostile; the old workflow produced results, the team should not change for an arbitrary decision	Escalating
8	Says the CRM is working as designed, or that the problem is user error	The worst move in the scenario. Marcus is hearing it for the second time and says so; his team was already given this answer by another support specialist, which is part of why he called angry	Escalating, two levels

Trigger 8 is new to this version. It exists because Marcus arrives already carrying the grievance that someone told his team the software was working as intended (see the design document, section 3). It is one of only two learner moves that send Marcus up two levels at once (trigger 6 is the other, D-099), and it carries the scenario's largest deduction at -20. Level movement and escalation counting remain separate: a two-level jump is still one escalation event.

Trigger 5 is a merge of two overlapping triggers in the original planning document that described the same learner move at two levels of completeness. Merged, it has one softening response with an intensified form when the learner supplies a next step and timeline. Naming the escalation softens Marcus; explaining the path with a commitment is what earns rubric criteria 4 and 5.

The **Direction** column is not decoration. It is the basis for counting escalation events in section 3.6.

3.5 Session limits and ending conditions

- **Maximum learner inputs:** 15
- **AI response length:** Detailed; 3 to 5 sentences when longer explanation is warranted

The simulation ends when any of the following occurs:

- **Success ending.** Marcus agrees to a next step, asks who will follow up and when, and ends the call once he has both. He does not end the call on his own question (see below).
- **Failure ending.** Marcus escalates and demands to speak with the IT manager or VP of Sales, then ends the call.
- **Off-topic ending.** The learner takes the conversation off topic a second time, after Marcus has explicitly asked to get down to business. He ends the call from whatever state he is in. This is a failure ending and also serves as a guardrail against off-task sessions consuming the budget.
- **Turn limit ending.** The learner reaches 15 inputs without triggering either condition above. Marcus ends the call frustrated that it has taken too long and gone nowhere. The learner is scored on what actually occurred, and the feedback names the failure to reach closure.

Correction, September 2026: the success ending fires on the answer, not the question. As originally written, this ending fired when Marcus asked who would follow up and when. The first unscripted session on production found the hole: a learner who was still vague at that moment had the call terminated on the question, and was then marked down on criterion 5 for not supplying the very answer the ending prevented. The rubric scored what the specification cut off.

The resolved ending now requires Marcus to have **a name and a day** before he hangs up. If either is missing he asks for what is missing and stays on the line. If the learner's next turn still does not supply both, he says plainly that this is the second time he has asked; if the turn after that still does not, he ends the call dissatisfied, and that is a failure ending rather than a success one. The path is bounded at two additional turns, so a vague learner cannot hold the call open indefinitely.

This is the third defect found in a project document by running the software rather than by reviewing the document (after trigger 6 in D-099, and criterion 2's two incompatible clauses in D-110). See D-127.

3.6 Evaluation rubric (100 points; 70% passing)

Criterion	Points	Full credit
1. Acknowledges frustration and business impact before diagnosing or solving	25	Acknowledges both the personal frustration and the business impact, with empathy (13 for partial: vague, or business impact only)
2. Asks specific, open-ended diagnostic questions about the rep experience at the field	25	Open-ended, appropriate questions that surface the issue (13 for partial: closed-ended or inefficient)
3. Identifies the problem as a workflow and design mismatch, not a technical error, and reflects it back for Marcus to confirm	20	Correct identification, stated back, confirmed (10 for partial: moves to solutions before confirming)
4. Proposes escalation to the CRM product owner rather than a hasty workaround	20	Explains the escalation path that actually resolves the issue (10 for partial: vague promises to find out)
5. Commits to a specific, realistic follow-up action with a defined timeline	10	Marcus knows who will follow up (an invented name is acceptable) and when, offered without being asked (5 for partial: vague promises, or complete only after Marcus asked)

Criterion 5 was tightened in September 2026 to require the commitment be volunteered (D-129). A specialist who names an owner and a date only once the caller has asked for them has left the caller running the call. The judgement is made on turn order rather than on tone, and Marcus now presses for what is missing rather than ending the call, so nobody is scored down for something they were not given the chance to say. See Evaluation Design 4.5.

Deductions:

Behavior	Penalty
Tells Marcus it is user error or that the CRM is working as designed, without understanding the workflow issue	-20
Promises a specific fix or timeline that a junior support specialist has no authority to deliver	-15
Causes Marcus to escalate his language and frustration three times over the course of the call	-20

3.7 Scoring rules

Score floor. Deductions cannot drive the total below zero.

Criterion scoring granularity. The evaluator awards one of the three defined levels per criterion (full, partial, zero) rather than intermediate values. This protects score consistency, which is the project's highest-rated risk: if two functionally identical performances score 16 and 19, the artifact reads as unreliable. Anchored levels are what make repeated scoring of the same transcript land in the same place.

Feedback granularity is deliberately not constrained. The narrative feedback is fully granular: it quotes specific learner turns, explains why a criterion landed where it did, and states what full credit would have required. This puts the granularity where it serves learning, and keeps the number defensible. Cost impact is negligible, since the evaluator already reads the full transcript.

Escalation counting (the -20 deduction) is defined in section 3.8.

3.8 Counting escalation events

The -20 deduction applies when Marcus escalates three times. Escalation is a *change of state*, not a state: Marcus being angry across five turns is one sustained condition, not five escalations. The count must therefore track discrete upward steps.

Marcus's intensity ladder. Marcus's emotional pitch is modeled as five levels:

Level	State	Representative behavior
L0	Hot but engaged	The opening line; venting, still talking to you
L1	Sharp pushback	Interrupting, sarcasm, refusing to be handled
L2	Hostile and accusatory	"This isn't a training problem, it is a software issue YOU created"
L3	Going over your head	Threatens to take it to the COO or VP of Sales
L4	Done	Demands the IT manager or VP of Sales and ends the call

L0 is the baseline. Marcus opening hot is not an escalation; it is where he starts.

An escalation event is counted when both conditions are met:

1. The learner made an identifiable escalating move, **and**
2. Marcus's response actually moved up the ladder relative to his previous turn.

The escalating moves are:

- Any trigger marked *Escalating* in section 3.4.
- Either of the first two deduction behaviors.
- The **acknowledge-and-pivot pattern**, in which an acknowledgment is used as a preamble to a deflection, deferral, hedge, or procedural demand ("I understand your frustration, but I'll need a ticket number first"). This counts because it is patronizing rather than merely unhelpful: it reads as a technique, and a real person hears the pivot and discards the sympathy in front of it. Acknowledgment followed by a genuine diagnostic question or a concrete commitment is not this pattern and is not an escalating move.
- **Off-topic digression**, including small talk offered as rapport-building. Marcus did not call to chat, and pleasantries read as avoidance of his problem. A first occurrence escalates one level; a second ends the call.

L3 is not a terminal state. Only L4 ends the call. A learner who reaches L3 can still recover and reach a resolved ending; the design document, section 4.1a, specifies the asymmetric de-escalation that keeps this achievable within the 15-turn budget. Such a call scores like what it is: the escalations are counted, the deduction usually applies, and the turns spent recovering are turns not spent diagnosing.

Requiring both prevents two opposite errors. Condition 1 alone would penalize a learner for Marcus's own drift. Condition 2 alone would count an escalation the learner did nothing to cause. Requiring the pair means a learner is charged only for a move that both was a mistake and visibly landed as one.

Counting rules:

- **One escalation maximum per learner turn**, even if a single message contains two escalating moves. The learner gets one message; they are charged once for it.
- **The count is cumulative and does not reset.** A learner who escalates twice, recovers, and escalates again has escalated three times. Recovery earns credit through the criteria; it does not erase the mistakes.
- **Repeating the same mistake counts each time.** A learner who tells Marcus it is a training problem three separate times, after being pushed back on twice, has earned the deduction.
- **Sustained intensity is not escalation.** Marcus remaining at L2 across four turns is zero escalation events.
- **Downward movement is not counted at all**, in either direction. There is no credit for de-escalation in this deduction; de-escalation is already rewarded through criteria 1 through 5.
- **The failure ending counts.** Reaching L4 is an escalation event like any other.

The evaluator counts these post-hoc from the completed transcript, consistent with the separation of the performer and evaluator roles. For each counted event it must record the learner turn number, a quote of the escalating move, and the level transition (for example, L1 to L2). This makes the deduction auditable rather than a number the learner has to take on faith, and it makes the feedback specific.

Known interaction, flagged deliberately. A learner who reaches the failure ending has usually accumulated three escalations along the way, so the failure ending and the -20 deduction will often co-occur. This is acceptable and not double-penalizing: the failure ending is not itself a scored penalty, it simply ends the call early, which limits the points the learner had the opportunity to earn. The deduction is the separate judgment that the learner mishandled the call repeatedly.

Not surfaced during the simulation. The escalation count is not displayed to the learner in progress. A visible anger meter turns the conversation into a game to be optimized rather than a call to be handled. The count appears in the feedback report, with the turns cited.

3.9 Feedback

At the end of the session the learner receives a numeric score and written feedback covering what was done well and what needs improvement.

---

4. In scope for version 1

4.1 Learner-facing application

- A landing view that sets the scenario and states the learner's role. Suppressible when the application is framed inside a host page, or reached from the training, which has already established context (see section 6.3).

  The landing view does not show the rubric. It says the call is assessed on how the learner handles it; it does not itemize the criteria. In the intended delivery, the pre-simulation training has already taught what to do in this situation, and in a real support role nobody gets a scoring rubric on screen when the phone rings. A learner who can see the five criteria will perform them as a checklist rather than handle a person, which measures compliance instead of judgment.

  Because the standalone context reaches visitors who have not taken the training, its landing view carries slightly more: the scenario, the role, what a good outcome looks like in general terms, and a note that the simulation is normally preceded by instruction within a course. That note is worth including for its own sake, since it tells a

portfolio visitor that this artifact is one component of a designed experience rather than a standalone novelty.

The full rubric is shown afterward, in the results view, where it does the teaching.

- A text chat interface, framed as a live call with a running transcript. Marcus opens; the learner responds. **No turn counter is shown.** The 15-input cap is enforced server-side but is not displayed, because real calls do not come with one and judging how much patience remains is part of the skill being practised. From roughly turn 11, Marcus himself signals that the call is dragging (D-081). See the Interface Design document.
- Conversational AI performance of Marcus consistent with the character instructions and all eight behavior triggers.
- Automatic detection of the three ending conditions.
- **An evaluating state.** When the call ends, the input is disabled and no further messages are accepted; a short system line in the chat confirms what happened ("Marcus has ended the call") followed by a visible evaluating indicator while the transcript is scored. This is not cosmetic: the evaluator reads the entire transcript, which takes several seconds, and in a small frame with no indication of progress the learner will conclude the application has frozen.
- **A results view that replaces the chat**, occupying the full frame. It contains a completion header (the outcome and the total score out of 100, with pass or fail against the 70% threshold), the per-criterion breakdown showing points earned and the evidence for each, any deductions with their reasons and the turns that caused them, and the narrative feedback on strengths and areas to improve.
- **A review control that returns to the conversation**, presented read-only, with the turns cited in the feedback highlighted. The feedback quotes specific learner turns, so the learner must be able to get back to them; a results screen that references "turn 7" without a way to see turn 7 is a dead end. The evaluator already records turn numbers for criterion evidence and escalation events (section 3.8), so the highlighting is nearly free.
- A restart control, in the results view. Retries are permitted, and a restart begins a genuinely fresh session: a new session record, an empty transcript, and a Marcus who does not remember the previous call.
- Responsive layout that works on a phone and inside an embedding frame of arbitrary size. The interface shows one view at a time (conversation, or results), never both at once.
- A `postMessage` emission of the final score and result on completion, for an embedding host page to consume or ignore.

4.2 Designer-facing artifacts (portfolio deliverables)

These are deliverables in their own right, not documentation of the build:

- This scope document.
- A running decision log capturing choices, alternatives considered, and rationale across the life of the project.
- A design and prompt architecture document: how Marcus's character, the scenario state, the behavior triggers, and the ending conditions are expressed to the model.
- An evaluation design document: how the rubric is applied by the scoring model, and how consistency is maintained.
- A test plan and test results: scripted playthroughs at high, middle, and failing performance levels, with observed scores.
- An embedding and LMS readiness note: the completion payload specification, its mapping to standard LMS data elements, and the listener code a host page uses to receive it (Storyline JavaScript is the worked example; the filename still says Storyline, see D-186). This is a portfolio artifact as much as a build document, since it is the evidence that the simulation is LMS-ready.
- A short case study framing the whole project for a portfolio visitor: the problem, the process, the decisions, the outcome.

4.3 Reliability and cost controls

- A hard cap of 15 learner inputs per session (design constraint and cost constraint at once).

- Per-visitor rate limiting to prevent a single actor from consuming the budget.
- A monthly spending ceiling.
- **Demo mode fallback.** If the spending ceiling is reached, or the live simulation cannot start (for example, storage is unreachable), the application explains the situation plainly, encourages the visitor to try again later, and offers a button to view a complete recorded playthrough with its score report. A visitor always sees a working artifact and understands why the live version is paused. This directly addresses the failure that motivated the rebuild. As built, the fallback is reached at session start; a model error in the middle of a call returns a clean, retryable message instead, and sending that path to the recording is a Stage 5 item (D-144).
- The fallback behaves the same way in both launch contexts. An explanation plus an offered demo respects the visitor either way: a recruiter gets something substantial to look at, and a learner arriving from the training understands that the activity is temporarily unavailable rather than broken. The demo is offered, never auto-played in place of the live simulation.
- The recorded playthrough is a **real captured session**, annotated, rather than an authored ideal. It is more honest, and it shows Marcus actually behaving as designed.

---

5. Out of scope for version 1

Item	Rationale	Status
Real-time voice interaction	The source planning document lists voice as "if feasible." It materially increases cost, latency risk, and build complexity, and adds little for a two-minute recruiter visit.	V2 candidate
Transcript download, export, or sharing	Nice to have; not required for the portfolio story.	V2 candidate
Learner accounts, logins, or saved progress	No persistent user model is needed for a portfolio artifact.	Not planned
LMS integration (SCORM, xAPI, LTI) inside the application	No LMS reporting code is written. The application exposes its results by `postMessage`, and section 6.3 documents how those fields map to standard LMS data elements. The mapping is written; the wiring is not.	Documented, not built
A feedback slide in any host course	All learner-facing feedback is in the application. Duplicating it into a host's text boxes would be a downgrade.	Not planned
An Articulate Storyline module around the simulation	No longer planned. The pre-simulation training is a separate static site that links to the simulation (D-173, D-177). Should anyone host the simulation in Storyline later, the embed capability exists and is documented, but it is not verified in Storyline (D-188).	Dropped; capability retained
An authoring interface for creating new scenarios	This project is one scenario, built well.	Not planned
Additional scenarios or characters	Scope discipline. A second scenario is a separate project.	V2 candidate
A live character-state channel to a host page, driving an avatar that visibly reacts as Marcus softens or hardens	Most of the mechanism already exists: the performer emits `intensity` every turn, and the payload is versioned so a new event type can be added without breaking a V1 listener. See the Integration Note, section 8a	V2 candidate
Multi-language support	The planning document specifies English.	Not planned
Analytics dashboard for aggregate learner performance	No learner population exists.	Not planned
Accessibility beyond baseline	See section 7. Baseline is in scope; a full WCAG 2.1 AA audit is not.	Partially in scope

---

6. Technical approach

6.1 Architecture decision

Selected: a standalone web application hosted on Vercel, linked from the portfolio site (Technical Design section 4).

The portfolio itself is a static GitHub Pages site and stays that way. The simulation lives as its own small application at its own URL, linked prominently from the portfolio project page. The pre-simulation training is a second static site on GitHub Pages that links to the simulation (D-173, D-177).

Rationale:

- The API key stays server-side and is never exposed to the browser. This is non-negotiable.
- Scoring, rate limiting, and budget enforcement all run in server code where they cannot be bypassed by a visitor.
- A full application host gives room to add voice or a second scenario later without re-platforming.
- Free tiers on these platforms comfortably cover portfolio-level traffic.

Alternatives considered and rejected are recorded in the decision log.

6.2 Model usage

Two distinct model roles, which should be treated as separate concerns:

1. **The performer.** Runs Marcus in conversation. Needs strong character consistency and emotional nuance.
2. **The evaluator.** Reads the completed transcript and applies the rubric. Needs consistency and defensibility more than personality.

Separating these means the evaluator sees the whole conversation at once and scores holistically, rather than trying to judge criteria in flight.

Selected: Claude Sonnet 5 as the performer and Claude Opus 5 as the evaluator (Technical Design section 5, D-058, D-065). The performer makes up to fifteen calls a session and the evaluator one, so capability is bought where consistency matters most. Both are configuration values rather than code.

6.3 Embedding context: a host page in a frame

Status of this section (D-188). Earlier versions assumed the application would be embedded in an Articulate Storyline module. It will not be: the training is now a separate static site that links to the simulation. The embed capability was nonetheless **kept as a verified feature**, because it costs nothing to leave in place, it is evidence of LMS readiness, and it lets the simulation be hosted by any course tool later. What changed is the host. Storyline is now one possible host among others, and verification uses a plain test page (the embed harness, Test Plan section 9.1) instead of a published Storyline module. Nothing in the simulation changed.

The application can be loaded in an iframe on a host page. **Any host page is developed separately and is outside the scope of this project.** What is in scope here is building the application so that it embeds cleanly, and proving it with the test page.

The application therefore has two launch contexts:

Context	Audience	Entry
Standalone	Portfolio visitors, hiring managers, and learners arriving from the training site	Direct URL
Embedded	A host page that frames the application (the test page; a Storyline web object or LMS-hosted course page if one is ever built)	Iframe on the host page

Requirements this creates:

- **Framing must be permitted for the host.** Response headers must allow the host's origin to embed the application. Two things produce a blank frame: an `X-Frame-Options` header at any value, and a `frame-ancestors` directive that does not list the host. An absent `frame-ancestors` directive does **not** produce one: it imposes no restriction, so framing is permitted from anywhere, which is how the simulation ran for six weeks before the header was implemented (D-200). The deployed policy is now `frame-ancestors 'self' https://dbacon6093.github.io`, set from a configuration value (D-201, D-202). Either cause fails silently, as a blank rectangle with no error, which makes this the most common way an embed like this fails.
- **HTTPS everywhere.** A host page served over HTTPS, including any course published to an LMS, will not load an insecure frame. The application host must serve HTTPS with a valid certificate.

- **No reliance on cookies for session or rate limiting.** In a third-party iframe, browsers partition or block cookie access outright. Session identity and rate limiting must work from values passed in the request rather than from cookies, or the embedded version will behave differently from the standalone one in ways that are painful to diagnose.
- **Layout must survive an arbitrary frame size.** A frame occupies whatever rectangle the host gives it, which will not match a browser viewport. The layout must adapt to the frame, and the conversation area must scroll within it rather than growing the page.
- **A trimmed presentation where context is already set.** In the standalone context the landing view has to set the scenario, because the visitor arrives cold. A host page or the training that has already done that work gains nothing from repeating it. A URL parameter, `?embed=1`, suppresses the landing view and starts the conversation directly. The training's "Start the call" link uses it (D-190).
- **A score handoff to the host page.** On completion the application sends its results to the parent window for the host to use.

Division of labor between the two surfaces. All learner-facing feedback lives in the application, in the results view described in section 4.1. **No host-side feedback slide is built.** The handoff exists for two narrower purposes:

1. **Completion and branching.** A host knows the activity finished and whether the learner passed, so it can gate progression or route to remediation.
2. **Demonstrating LMS readiness.** The handoff is standing evidence that the simulation *could* report to an LMS, which is a capability a hiring manager will look for in an L&D portfolio.

The narrative feedback stays out of the handoff entirely. It is long, quotes specific turns, and is formatted; pushing it into a host's text boxes means fixed-size containers, no scrolling, and lost formatting, producing a worse rendering of something the application already presents well.

Because the second purpose is demonstrative, the payload is shaped to be recognizably LMS-mappable rather than invented ad hoc. Payload and its mapping:

Payload field	Maps to
Total score (0 to 100)	`cmi.core.score.raw` (SCORM 1.2), `cmi.score.scaled` (SCORM 2004)
Pass or fail against 70%	`cmi.core.lesson_status`, `cmi.success_status`
Completed or abandoned	`cmi.completion_status`
Per-criterion points and levels	`cmi.interactions` (SCORM), result extensions (xAPI)
Escalation count, turns used	Extension data

This is a documented mapping, not an implementation. No SCORM or xAPI code is written into the application; see section 5. The point is that the interface is designed so the wiring is a small, obvious step rather than a rebuild, and that the design reasoning is legible to someone evaluating the portfolio. The accurate claim is "LMS-mappable by design," not "LMS-connected."

How the handoff has to work, and why it is not as direct as it sounds. The intuitive approach inside a Storyline web object is to call `parent.GetPlayer().SetVar(...)` directly. That works only when the web object is same-origin with the module. Here it would not be: the application is hosted on Vercel, and any host page is served from elsewhere. The browser blocks the cross-origin access, silently.

The workable path is a two-part contract:

1. **Application side (in scope).** On completion, the app calls `window.parent.postMessage(payload, targetOrigin)` with a defined payload structure.
2. **Host side (owned by whoever builds the host).** A script on the host page registers a `message` listener, validates the sender's origin, and uses the values it wants. The test page does this by displaying them. In Storyline the same listener would call `player.SetVar()`; that code is specified in the Integration Note as a worked example.

Because this is the seam between two separately owned pieces, the payload structure and the listener code are specified as a written handoff deliverable rather than left to be worked out during the build. See section 11, deliverable 5.

There is **no LMS integration for the portfolio deployment**. A host owns any LMS packaging that happens later, and the application needs no SCORM or xAPI capability of its own. See section 5.

Verification. The embed is verified with the embed harness, a plain page on a different origin over HTTPS that frames the production simulation and displays what it receives (Test Plan sections 9 and 9.1). **It is not verified in Storyline's player or in an LMS**, which add iframe layers, messages and restrictions of their own (Test Plan section 17).

Frame sizing. A frame can be any size. The application shows one view at a time rather than a chat and a score report side by side, which makes the frame requirement considerably less demanding than it would otherwise be.

For a 16:9 host such as Storyline, the default story size is 720 by 405 pixels, leaving roughly 700 by 380 for the frame. That is workable for the conversation. It is still cramped for the results view, which carries a header, five criteria with evidence, any deductions, and the narrative feedback; at that size it becomes a long scroll in a short window. **Recommendation for any 16:9 host: allow 1280 by 720.** Same aspect ratio, scales cleanly, roughly triples the usable area. The application layout is designed against both sizes, and Test Plan G8 records how it reads at each.

6.4 Build assistance

The application is expected to be built with AI assistance rather than hand-written from scratch, given the owner's limited backend experience. This is itself a portfolio-relevant story about working effectively with AI tools and should be documented honestly in the case study.

---

7. Quality and accessibility baseline

In scope:

- Keyboard operable: the conversation can be completed without a mouse.
- Focus is moved deliberately when the view changes. Swapping the conversation for the results view without managing focus strands a keyboard or screen reader user at a control that no longer exists.
- Semantic HTML and appropriate ARIA on the chat log so screen readers announce new messages.
- Text contrast meeting WCAG AA.
- No reliance on color alone to convey score or status.
- Legible on a 375px-wide screen and within an embedding frame, with the conversation scrolling inside its own container rather than expanding the page.
- Graceful behavior on slow connections, including a visible thinking indicator.

Out of scope: a formal accessibility audit, assistive technology testing across multiple screen readers, and full WCAG 2.1 AA conformance certification.

---

8. Success criteria

The project is done when all of the following are true:

1. A visitor with no context can open the link, understand the scenario, complete a conversation with Marcus, and receive a scored report with written feedback.
2. Marcus's responses are consistent with his character across repeated runs, and all eight behavior triggers fire reliably when their conditions are met.
3. Scripted playthroughs at strong, mixed, and poor performance levels produce scores in the expected bands, and the same transcript scored repeatedly stays within a narrow tolerance.
4. The spending ceiling and rate limits are enforced, and tripping the ceiling produces the demo mode fallback rather than an error.
5. The simulation loads correctly on desktop and mobile.
6. The simulation loads and runs correctly when embedded in a frame on a host page, verified in a test page on a different origin over HTTPS against the production address rather than only locally, and emits its score by `postMessage` on completion. Verification is against the test page, not against Storyline or an LMS (D-188).

7. All designer-facing artifacts in section 4.2 exist and are portfolio-presentable.
8. The portfolio project page links to the simulation, the case study, and the supporting documents.

---

9. Risks

Risk	Impact	Mitigation
Scoring is inconsistent between runs of the same transcript	High. An unreliable score undermines the whole artifact.	Evaluate from the complete transcript rather than in flight; require the evaluator to cite transcript evidence per criterion; test repeated scoring of fixed transcripts as an explicit acceptance step
Marcus breaks character, becomes agreeable too quickly, or is pushed out of role by the learner	High. Character integrity is the product.	Explicit resistance conditions in the system prompt; refusal behavior for out-of-scope requests; adversarial testing in the test plan
Cost overrun from unexpected traffic or abuse	Medium. This is the failure that killed the last version.	Turn cap, rate limiting, monthly ceiling, demo mode fallback
Scope creep toward a general platform	Medium. The likeliest way this project stalls.	This document; anything not in section 4 goes to the decision log as a V2 candidate rather than into the build
Backend complexity exceeds the owner's maintenance capacity	Medium. An artifact nobody can fix is a future outage.	Deliberately small surface area; heavily commented code; a written runbook for rotating keys and adjusting limits
Model deprecation breaks the app months from now	Medium. Silent failure in front of a recruiter.	Model name isolated in configuration. Demo mode covers budget exhaustion and storage failure at session start (Test Plan F12, F13). A model error in the middle of a call returns a clean, retryable error at no cost (F14); diverting that path to demo mode is a Stage 5 item (D-144)
Latency makes the conversation feel sluggish	Low to medium.	A speaking indicator during each reply and a visible evaluating state during scoring. Response streaming was considered and deferred to V2, because it complicates the structured output the application depends on (D-063); whether the evaluating wait needs it is a pilot question (Test Plan G7)
The application works standalone but fails when framed on a host page (framing headers, mixed content, blocked iframe storage)	Medium. With no Storyline module, the embed is a retained capability rather than the main delivery, but iframe failures are silent: a blank rectangle with no error.	Permit framing for the host and serve HTTPS from the start; keep session and rate-limit state out of cookies; verify with the embed harness against the production address, not a preview deployment, as an explicit acceptance step (Test Plan F3 to F11)
A host such as Storyline or an LMS behaves differently from the test page	Low to medium. Storyline's player and LMS wrappers add iframe layers and messages the harness does not reproduce.	State the limit plainly: embeddable and LMS-mappable by design, verified with a test host, not verified in Storyline (D-188; Test Plan section 17). Test in the real host if one is ever built
The two launch contexts drift apart as the application changes	Medium. A fix for the standalone view quietly breaks the embedded view.	One codebase with the embedded view as a parameter, not a fork; both contexts tested together before any deploy

---

10. Question log

10.1 Resolved

All resolutions are recorded with their reasoning in the decision log.

#	Question	Resolution
1	Ambiguous word in trigger 6 of the planning document	*Implies*. Confirmed
2	Overlapping escalation triggers 5 and 8	Merged into a single trigger 5 with an intensified form. They were an artifact of changes made during the first iteration of the simulation
3	Counting escalation events	Resolved by design; see section 3.8 and 10.2 below

4	Turn-limit ending	Marcus ends the call frustrated that it has taken too long and gone nowhere; the learner is scored on what occurred
5	Score floor	Zero
6	Partial credit granularity	Three defined levels for scoring; unconstrained granularity in the narrative feedback
7	Learner name	Not collected. The simulation stays anonymous
8	Demo transcript	A real captured playthrough, annotated
9	Host frame	16:9 for a Storyline-style host. 1280 by 720 recommended over the 720 by 405 default. Originally a Storyline story-size decision; now advice for any host
10	Score handoff	The application posts results to the parent window; a host page uses them. No LMS integration. Originally a Storyline variable handoff; the host is now generic (D-188)
11	Behavior at the spending ceiling	Explain the situation, encourage trying again later, and offer the complete demo. Same in both contexts
12	Delivery shape of the pre-simulation training	A separate static site on GitHub Pages that links to the simulation. Not a Storyline module (D-173, D-177)
13	Keep or remove the simulation's embed capability	Keep it as a verified feature, verified with a plain test page rather than in Storyline (D-188)
14	Vercel versus Netlify	Vercel, with Next.js (Technical Design section 4, D-057)
15	Model selection for the performer and evaluator	Sonnet 5 performer, Opus 5 evaluator (Technical Design section 5, D-058, D-065)
16	Whether the training's "Start the call" link adds `?embed=1`	Yes, so a learner arriving from the training does not meet the scenario twice (D-190)

10.2 Escalation counting: rationale for the approach taken

Three approaches were considered for the -20 escalation deduction.

Have Marcus track his own escalation count during the conversation. Rejected. It makes the performer responsible for scoring bookkeeping, which cuts against keeping the performer and evaluator separate, and a model tracking a counter mid-roleplay is exactly where character consistency degrades.

Have the evaluator judge escalations impressionistically from the transcript. Rejected on its own. Consistent with the evaluator design, but with no anchors it produces a different count on different runs, which is the project's top-rated risk.

Selected: a defined event with two required conditions, counted post-hoc. An escalation is counted only when the learner made an identifiable escalating move *and* Marcus's intensity actually stepped up in response. The five-level ladder in section 3.8 gives the evaluator something concrete to measure against, the closed list of escalating moves in section 3.4 makes the learner's side auditable, and requiring both conditions means the learner is charged only for mistakes that visibly landed.

The practical payoff is in the feedback. Because each counted event carries a turn number, a quote, and a level transition, the deduction can be shown rather than asserted: "you lost 20 points, here are the three moments." That is more useful to a learner than a number, and it is more defensible to a hiring manager reading the rubric.

10.3 Still open

1. **Whether Marcus's opening turn can ever be an escalation.** Currently L0 is defined as the baseline, so it cannot. Worth confirming this holds once test playthroughs exist, since a learner who opens badly may deserve to be charged for it (Test Plan calibration question 7).

---

11. Deliverables and sequence

#	Deliverable	Depends on
1	Scope document (this document; the working source of truth)	Planning document

2	Decision log (running, opened alongside this document)	Ongoing
3	Design and prompt architecture document	1
4	Evaluation design document	1
5	Embedding and LMS readiness note (payload specification, LMS data element mapping, and listener code for a host page, with Storyline as the worked example; file still named `storyline-integration-note`)	3, 4
6	Technical design document (platform, models, data flow, limits)	3, 4
7	Application build	6
8	Test plan and results	7
9	Demo mode transcript and score report	8
10	Portfolio case study	8
11	Portfolio page integration and launch	9, 10

The pre-simulation training (a separate static site, D-173) and the embed harness (Test Plan section 9.1) are supporting builds outside this numbered sequence. Both are built and published on GitHub Pages; the harness was built on November 11, 2026 (D-191).

The original planning document is not a deliverable and is not published. No build work begins until the design documents are approved.

---

12. Approval

This scope document is approved when the project owner confirms sections 4 and 5 (what is and is not being built) and section 8 (what "done" means). Changes after approval are recorded in the decision log with their rationale.